

Final Exam Topics

10. Fundamentals of programming languages

Note: This document was translated from Hungarian to English by AI. Please verify the information against the original Hungarian source before relying on it.

Fundamentals of programming languages

Compilation and linking, the rule system of a programming language. Lexical elements, syntax, semantic rules. Rules of evaluating expressions. Statements, control constructs. Representation of basic types. Composite types. Program structure, scope, visibility. Representation of variables in memory, lifetime. Parameter passing. Exceptions.

1 Compilation and interpretation

1.1 Compilation

During compilation, generally machine code is produced from a high-level programming language, which the processor is already able to interpret and run. Its advantage is that it is fast, since the lexical, syntactic, and semantic analysis runs once, at compile time, and at this point we also optimize the code. At compile time many errors can be filtered out, thereby making debugging easier. Machine code is hard to reverse-engineer. It is generally used for larger programs, where efficiency is important. The compiled code can no longer be changed later (or only with great difficulty).

Its disadvantage is that the resulting code is not platform-independent; it has to be compiled separately for every architecture.

Examples: C, C++, Ada, Haskell

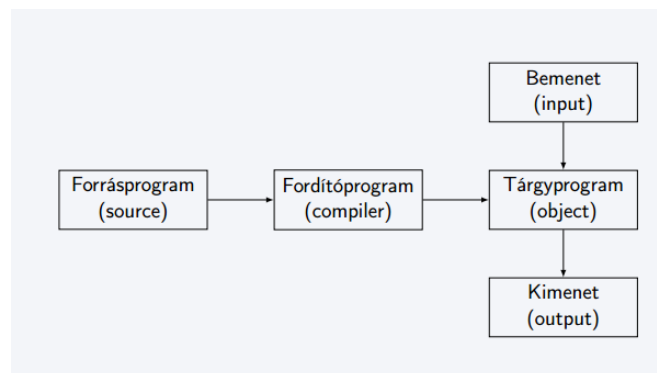


Figure 1: the process of compilation

1.2 Interpretation

During interpretation the interpreter executes the program code during run time. It is platform-independent; only the interpreter has to be written once for each system. Debugging in it is hard, since many errors can remain in the code that a compiler would have filtered out (e.g. type matching).

Examples: PHP, JavaScript, ShellScript

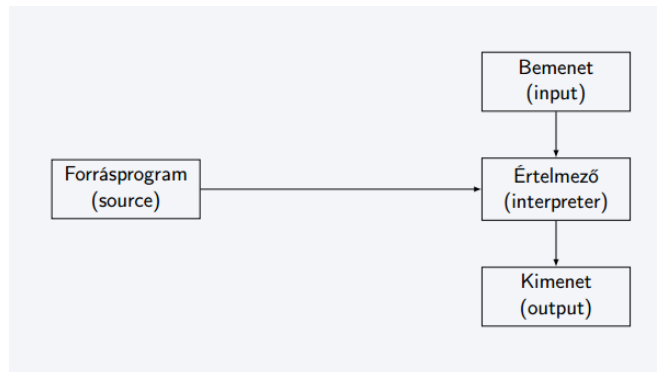


Figure 2: the process of interpretation

1.3 Compilation and interpretation together

Some languages (e.g. Java) use pre-compilation, whose result is the *bytecode*, which is machine code for a virtual machine. With this, compile-time error checking and optimization can be achieved, but platform-independence is preserved.

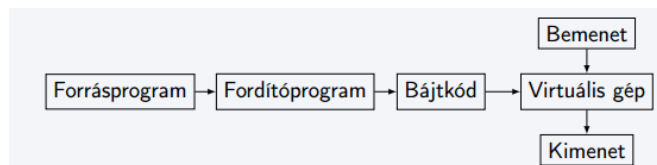


Figure 3: the process of interpretation

2 Compilation unit and linking

The creation of the object code happens in two phases. First we *compile* the source files; from this the so-called *object code* (e.g.: .obj, .class) is produced. In this the machine instructions are already present, but the references (e.g. variables, functions) that are implemented in other files are missing from it. We call *compilation unit* that from which one object code is produced.

The task of the *linker* is to fill in the missing references, so that, generating a single file, we get runnable code.

Linking can be static, when the compiler fills in the missing references; or dynamic, when it loads the missing code at run time, typically from another file (e.g.: .dll). The latter is practical when a module is used by several separate programs.

3 The components of the compiler

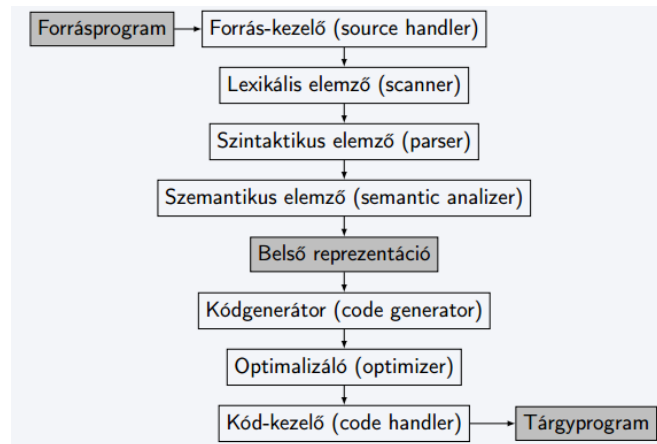


Figure 4: the steps of compilation

3.1 Lexical analyzer

Its input is the source code itself. The task of the lexical analyzer is to break the source code into tokens. A regular (type-3) grammar, characteristic of the language, is given. This specifies what types of tokens can appear in the source. It can assign properties to the tokens (e.g. the name of a variable, the value of a literal). Its output is this token sequence. If the analyzer finds a character sequence to which no token can be assigned, then this triggers a lexical error.

remark: For a lexical error the compilation process does not necessarily break off; we can try to skip the given part and continue the analysis, so that if there are several errors, we can report them at once.

We recognize the regular expressions with *deterministic finite automata*. If, for a lexical element, one of the automata gets into an accepting state, then we have recognized a token. A character sequence can be recognized by several automata at once. If these are equally long, then the language is conflicting. This must not occur. It is, however, possible that an automaton recognized a word and also its prefix. Then we always choose the longer one.

3.2 Syntactic analyzer

Its input is the output of the lexical analyzer. Its task is to build a *syntax tree* from the tokens, based on a context-free (type-2) grammar belonging to the language, or, if this is impossible, to report it as a *syntax error*.

3.2.1 LR0 analysis

We read the symbol sequence produced by the lexical analyzer from left to right, and put the symbols onto the analyzer's stack.

Shift: we put a new symbol from the input onto the top of the stack.

Reduce: we replace the rule right-hand side on the top of the stack with the nonterminal standing on the left side of the rule

In the background a deterministic finite automaton works: the transitions of the automaton are determined by the symbols that get onto the top of the stack; if the automaton gets into a final state, we have to reduce, and in any other state we have to shift.

For certain languages the automaton can be conflicting: we cannot decide whether to shift or reduce.

3.2.2 LR1 analysis

It offers a solution to the previous problem, extending the set of possible languages.

The idea is to *look ahead* one symbol.

If the current state is i , and the result of the look-ahead is the symbol a :

if $[A \rightarrow \alpha.a\beta, b] \in I_i$ and $read(I_i, a) = I_j$ then we have to shift, and step into state j .

if $[A \rightarrow \alpha., a] \in I_i (A \neq S')$, then we have to reduce according to the rule $A \rightarrow \alpha$.

if $[S' \rightarrow S., \#] \in I_i$ and $a = \#$, then we have to accept the text; in every other case we have to report an error.

If in state i A gets onto the top of the stack: if $read(I_i, A) = I_j$, then we have to step into state j ; otherwise we have to report an error.

3.2.3 Legend / Canonical sets

Closure

If I is an $LR(1)$ item set of the grammar, then $closure(I)$ is the narrowest set that has the following properties:

$I \subseteq closure(I)$, if $[A \rightarrow \alpha.B\gamma, a] \in closure(I)$,

and $B \rightarrow \beta$ is a rule of the grammar, then for $\forall b \in FIRST1(\gamma a)$ we have $[B \rightarrow \beta, b] \in closure(I)$

Read

If I is an $LR(1)$ item set of the grammar, and X is a terminal or nonterminal symbol of it, then $read(I, X)$ is the narrowest set that has the following property:

If $[A \rightarrow \alpha.X\beta, a] \in I$, then $closure([A \rightarrow \alpha.X\beta, a]) \subseteq read(I, X)$.

LR(1) canonical sets (I_n)

- $closure([S' \rightarrow .S, \#])$ is a canonical set of the grammar.
- If I is a canonical item set of the grammar, X is a terminal or nonterminal symbol of it, and $read(I, X)$ is not empty, then $read(I, X)$ is also a canonical set of the grammar.
- With the first two rules all canonical sets are produced.

3.3 Semantic analyzer

Semantic analysis typically realizes the context-dependent checks.

- handling of declarations: variables, functions, procedures, operators, types
- visibility rules
- arithmetic checks
- the context-dependent parts of the program's syntax
- type checking
- etc.

For semantic analysis we have to extend the grammar. Let us assign attributes to the symbols and actions to the rules! The conditions belonging to a given rule can depend only on the attributes occurring in the rule. (If a condition does not hold, then a semantic error has to be reported!). The semantic routines can use and compute only the attributes of the rule to which the action symbol representing them belongs. In every syntax tree, every attribute value can be determined by exactly one semantic routine. The grammar created this way is called an *attribute translation grammar* (ATG).

The *well-defined attribute translation grammar* is an attribute translation grammar for which it is true that, in every syntax tree belonging to the sentences of the language defined by the grammar, the value of every attribute can be unambiguously computed.

An attribute can be determined in two ways:

By synthesis

the information spreads bottom-up in the syntax tree; we compute a parent's attribute from the children. We call distinguished those attributes that the lexical analyzer provides.

By inheritance

the information spreads top-down in the syntax tree. The attributes of the children are determined by that of the parent.

The *L-ATG* is an attribute translation grammar in which, in every $A \rightarrow X_1X_2...X_n$ rule, the attribute values can be determined in the following order:

- inherited attributes of A
- inherited attributes of X_1
- synthesized attributes of X_1
- inherited attributes of X_2
- synthesized attributes of X_2
- ...
- inherited attributes of X_n
- synthesized attributes of X_n
- synthesized attributes of A

If our grammar satisfies this, then every attribute can be efficiently determined.

For semantic analysis we typically use a symbol table, with a stack structure and a search tree or hash table. Every block is a new level in the stack; the search for a symbol starts from the top of the stack.

4 Rules of evaluating expressions

Concepts:

- Operands: Variables, constants, function and procedure calls.
- Operators: Operation signs that connect the operands in an expression and denote some operation.
- Expression: a sequence of operators and operands
- Precedence: Determines the order of evaluation of the operations.
- Direction of associativity: In expressions containing operators of the same precedence, the direction of evaluation. We distinguish left-associative and right-associative operators.

We can write the operators relative to the operands in three ways:

- Infix : An operator has to be written between its two operands (so only the operators of two-operand operations can be written this way). When several operators appear in an expression, then the order of execution of the different operators is decided by the precedence of the operators. The operator whose precedence is higher (e.g. that of multiplication is higher than that of addition) – the operation it denotes is evaluated first. The order of execution of the same operators is decided by the direction of associativity (e.g. left-associative means that it has to be executed proceeding from left to right). These (built into the programming languages) rules can be overridden with the help of parentheses.

Example: $A * (B + C) / D$

- Postfix (Polish notation) : We write the operators after their operands. The order of evaluation always happens from left to right – so an operator with n operands applies to the first n operands to its left.

Example: $A \ B \ C \ + \ * \ D \ /$

The same parenthesized (unnecessary): $((A \ (B \ C \ +) \ *) \ D \ /)$

- Prefix : We write the operators before their operand. The order of evaluation happens from left to right.

Example: $/ \ * \ A \ + \ B \ C \ D$

The same parenthesized (unnecessary): $(/ \ (* \ A \ (+ \ B \ C) \) \ D)$

Although for prefix operators too the evaluation happens from left to right, if after an operator another operator follows, then obviously the operation belonging to this operator has to be executed first, so that we can also execute the left-hand one. In the above example too, the multiplication has to be done before the division, and the addition before the multiplication.

Evaluation of expressions containing logical operators

There are two kinds of evaluation of such expressions:

- Lazy evaluation : If the value of the expression can be determined from the first argument, then it does not evaluate the second.
- Eager evaluation : It determines the logical value of both arguments in any case.

The two evaluation methods can in certain cases give different results:

- The 2nd argument is not always meaningful

Example (C++):

```
if ((i>=0) && (T[i]>=10))
{
    //...
}
```

Suppose that T is an `int` array, indexed from 0. Here if $i \geq 0$ is false, then we would under-index T. This, in the case of eager evaluation, would cause a run-time error. In the case of lazy evaluation (C++ uses this by default), however, we can know that the condition can no longer be true, so $T[i] \geq 10$ no longer has to be evaluated.

- The 2nd argument has some side effect.

Example (C++):

```
if ((i>0) || (++j>0))
{
    T[j] = 100;
}
```

In this case, if $i > 0$ is true, then the condition is certainly true, but the expression $++j > 0$ has a side effect: it increases the value of j. Since C++ uses lazy evaluation for the `||` operator (for the `|` operator the evaluation is eager), in this case it does not increase the value of j. (only if $i > 0$ is false).

5 Statements, control constructs

5.1 Simple statements

- **Assignment** : On the left side of an assignment there can be a variable, on the right side any expression. With the assignment we assign the right-side expression to the variable. One has to take care that the right side has an expression appropriate for the type of the left-side variable (or there exists an implicit conversion, e.g. in C++ between the integer and logical types). In most languages the operator of assignment is = (for example C++, Java, C#), or := (for example Pascal, Ada).
- **Empty statement** : One cannot write such a thing everywhere. Its essence is that it does nothing. It can have a *raison d'être* in languages where we cannot write an empty block, where at least 1 statement must appear in it. (e.g. Ada) The empty statement serves this purpose. (In Ada this is the `null` statement)
- **Subprogram call** : We can call subprograms by giving their name and parameters. Examples:

```
– System.out.println("Hello");
– int x = sum(3,4);
```

- **Return statement** : The effect of the statement is that the execution of the subprogram ends. If the subprogram is a function, then the return value also has to be given.

Examples (C++):

- In this example `doSomething` is a procedure, it has no return value. The parameter `x` is assigned to `j`. If this value is not 0, then we return, that is, we interrupt the execution of the subprogram (we do not return a concrete value, however). If this value is 0, then execution continues, we call the `doSomethingElse` function with the parameter `j`.

```
void doSomething(int x)
{
    int j;
    if(j=x)
        return; // j!=0, do nothing

    doSomethingElse(j);
}
```

- In this example `isOdd` is a function, with an `int` return value. It tells, about the parameter integer `x`, whether it is odd. For this it “ands” `x` with 1 (0...01) using bitwise AND. If the result is not 0, then it is odd, we return with the value `true`. Otherwise we continue operation, then return with the value `false`.

```
int isOdd(int x)
{
    if(x & 1) //bitwise AND with 0...01 is not 0...0
        return true;

    return false;
}
```

- **Statement block**: The statements inside the block “belong together”. This is a language element well usable in several cases:
 - In control constructs: We can separate the statements belonging to the given control construct.

- In languages where a declaration is possible only in the declaration blocks at the beginning of the program (e.g. Ada), it is possible to open a block in a later part of the program code, where declarations can also appear.
- Initializer block of classes, e.g. in Java (executed before the constructor):

```
public class MyClass{
    private ResourceSet resourceSet;

    {
        resourceSet = new ResourceSetImpl();
        UMLResourcesUtil.init(resourceSet);
    }

    /* ... */
}
```

- Static initializer block of classes, e.g. in Java:

```
public class MyClass{
    private static ResourceSet resourceSet;

    static{
        resourceSet = new ResourceSetImpl();
        UMLResourcesUtil.init(resourceSet);
    }

    /* ... */
}
```

5.2 Control constructs

- Branching : Branching is a control construct with which we can determine that certain (block-given) statements can be created only with the given condition. Generally we can give several conditions one after another (if L1 then ... else if L2 then ... else if L3 then ...). We can also specify what should happen if no condition holds (if L then ... else ...).

Branchings can be nested into each other (if ... then if ...).

“Dangling else” problem: It arises in languages where a statement block of a condition is not closed off with a separate keyword (e.g. endif). Then, if we nest branchings into each other, the following problem can arise: Example: `if (A>B) then if (C>D) then E:=100; else F:=100;`

In the above case it cannot be determined to which branching the programmer meant the else keyword.

- Loop : Means the some-number-of-times execution of a statement block (loop body).
 - Loop without condition : Codes an infinite loop; one can exit it with the unstructured statements (see below), or with return, or possibly in the case of an error arising.

Example (ADA):

```
loop
    null;
end loop;
```

- Pre-tested loop : Before every execution of the loop body, the loop examines whether the given condition holds. If it holds, then it executes the body, then checks again. Otherwise execution continues after the loop.

Examples:

* C++:

```
int x=0,y=0;
while(x<5 && y<5)
{
    x+=y+1;
    y=x-1;
}
cout<<x+y<<endl;
```

* ADA:

```
declare
    X,Y: Integer;
begin
    X:=0;
    Y:=0;
    while X<5 and Y<5 loop
        X:=X+Y+1;
        Y:=X-1;
    end loop;
end;
```

- Counting loop : In this control construct we can specify how many times the loop body should be executed. Example (ADA):

```
for i in 1..10 loop
    null;
end loop;
```

The counting happens by storing in a variable (loop variable) “where we are” – this we compare at the beginning of every loop with the expression that the variable (i) has to exceed. So it can also be regarded as a pre-tested loop, where the loop condition is $i \leq 10$ and at the end of the loop body i has to be increased.

- Post-tested loop : The difference compared to the pre-tested one is that we check the condition after the execution of the loop body – so here the loop body runs at least once in any case.

Example (C++):

```
int i=0;
do
{
    ++i;
} while(i<5);
```

Remark: there are languages (e.g. Pascal) in which the condition of the post-tested loop is not a staying-in but a stopping condition. That is, we do not specify what has to hold for the loop body to be executed once more, but what has to hold for the loop body not to be executed any more.

The Pascal counterpart of the previous C++ example:

```
i:=0;
repeat
    i:=i+1;
until i=5;
```

Remark: in ADA there is no real post-tested loop. In this language we can write a post-tested loop by writing a loop without a condition whose last statement is a condition-bound exit.

Example:

```
i:=0;
loop
    i:=i+1;
    exit when i=5;
end loop;
```

- **foreach** : Used when we want to execute the body for every element of a data structure. Essentially this too is a pre-tested loop (the loop condition is whether we have reached the end of the data structure).

Example:

```
foreach (int v in Vect)
{
    ++v;
}
```

Remarks:

1. Not every programming language has such a loop. It has spread mostly in the newer languages.
2. In not every programming language is **foreach** the keyword for this loop. For example in Java, in C++11 (in older versions there is no such thing) we can use the **for** loop as **foreach**:

```
int x=0;
for (int v : vect){
    x+=v;
}
```

5.3 Unstructured statements

- **Breaking out of a loop** : Can be used for “jumping out” of a loop (that is, its immediate termination). We often use it for breaking out of an infinite loop, or for coding a post-tested loop (where there is no built-in control construct for this, for example Ada).

Such a statement is e.g. **break** in C/C++/C# or Java, and **exit** in Ada.

- **goto statement** : We can define labels in the program code, then with the **goto** statement we can direct the control to such a label. Control constructs can also be coded with it. Its excessive use can lead to unreadable code.

5.4 Recursion

Recursive subprogram: A subprogram in whose code there appears a call to itself. The recursion must in any case have a stopping condition – that is, a condition (together with a branching) whose fulfillment means no recursive

call happens, so that all the in-progress recursive calls can be executed (otherwise infinite recursion arises, and then generally, sooner or later, the stack fills up and the program stops).

Examples (factorial):

- C++:

```
int fact (int n)
{
    if(n>0)
        return n * fact (n-1);
    else
        return 1;
}
```

- Haskell:

```
fact :: (Integral a) => a -> a
fact n
    | n>0 = n * fact (n - 1)
    | otherwise = 0
```

6 Code generation for basic control constructs

The task of code generation is to transform the syntactically and semantically analyzed program into object code. It is generally closely connected with semantic analysis.

6.1 Assignment

assignment $\rightarrow$ variable assignmentOperator expression

code evaluating the expression into the eax register
2 mov [Variable],eax

6.2 One-branch branching

statement $\rightarrow$ if condition then program end

```
1 code evaluating the condition into the al register
2 cmp al,1
3 je Then
4 jmp End
5 Then: code of the program of the then branch
6 End:
```

remark: the double jump is needed because the range of the conditional jump is limited.

6.3 Multi-branch branching

statement $\rightarrow$

if *condition*₁ then *program*₁

elseif *condition*₂ then *program*₂

...

elseif *condition*_{*n*} then *program*_{*n*}

else *program*_{*n*+1} end

1 evaluation of the 1st condition into the al register

2 cmp al,1

3 jne near Condition_2

4 code of the program of the 1st branch

5 jmp End

6

...

7 Condition_n: evaluation of the n-th condition into the al register

8 cmp al,1

9 jne near Else

10 code of the program of the n-th branch

11 jmp End

12 Else: code of the program of the else branch

13 End:

6.4 Switch-case

statement $\rightarrow$ switch variable

case *value*₁ : *program*₁

...

case *value*_{*n*} : *program*_{*n*}

1 cmp [Variable],Value_1

2 je near Program_1

3 cmp [Variable],Value_2

4 je near Program_2

5

. . .

6 cmp [Variable],Value_n

7 je near Program_n

8 jmp End

9 Program_1: code of the program of the 1st branch

10

. . .

11 Program_n: code of the program of the n-th branch

12 End:

6.5 Loop

6.5.1 Pre-tested

statement $\rightarrow$ while condition statements end

```

1 Start: evaluation of the loop condition into the al register
2 cmp al,1
3 jne near End
4 code of the program of the loop body
5 jmp Start
6 End:

```

6.5.2 Post-tested

statement $\rightarrow$ loop statements while condition

```

1 Start: code of the program of the loop body
2 evaluation of the loop condition into the al register
3 cmp al,1
4 je near Start

```

6.5.3 For loop

statement $\rightarrow$ for variable from $value_1$ to $value_2$ statements end

```

1 computing the "from" value into the [Variable] memory location
2 Start: computing the "to" value into the eax register
3 cmp [Variable],eax
4 ja near End
5 code of the loop body
6 inc [Variable]
7 jmp Start
8 End:

```

6.6 Static variables

Compilation of a variable definition without an initial value:

```

section .bss
; the previously defined variables...
Lab12: resd 1 ; an area of 1 x 4 bytes

```

Compilation of a variable definition given with an initial value:

```

section .data
; the previously defined variables...
Lab12: dd 5 ; the value 5 stored on 4 bytes

```

6.7 Logical expressions

6.7.1 expression1 < expression2

```

; evaluation of the 2nd expression into the eax register
push eax
; evaluation of the 1st expression into the eax register

```

```
pop ebx
cmp eax,ebx
jb Smaller
mov al,0 ; false
jmp End
Smaller:
mov al,1 ; true
End:
```

6.7.2 expression1 { and, or, not, xor } expression2

```
; evaluation of the 2nd expression into the al register
push ax ; one cannot push 1 byte onto the stack!
; evaluation of the 1st expression into the al register
pop bx ; it is in the bl part of bx,
; which is important to us
and al,bl
```

6.7.3 lazy "and" evaluation

```
; evaluation of the 1st expression into the al register
cmp al,0
je End
push ax
; evaluation of the 2nd expression into the al register
mov bl,al
pop ax
and al,bl
End:
```

6.7.4 Realization of subprograms

```
; evaluation of the 1st expression into the al register
cmp al,0
je End
push ax
; evaluation of the 2nd expression into the al register
mov bl,al
pop ax
and al,bl
End:
```

6.7.5 Calling subprograms

```
Scheme of subprograms
; evaluation of the last parameter into eax
push eax
; ...
```

```

; evaluation of the 1st parameter into eax
push eax
call subprogram
add esp,'total length of the parameters'

```

7 Code optimizer

The task of optimization is to make the resulting code smaller and faster, so that the result of the run does not change. Speed and compactness often contradict each other, and one can be improved only at the expense of the other. It is usually carried out in three steps:

- Performing optimization steps on the original program (or a simplified version of it)
- Code generation
- Performing machine-dependent optimization on the generated code

7.1 Local optimization

A sequence of consecutive statements in a program is called a *basic block* if, except for the first statement, control cannot be transferred from afar to any of its statements (in assembly programs: where the `jmp`, `call`, `ret` statements “jump”; in high-level languages: procedures, the start of loops, the first statement of the branches of branchings, the targets of `goto` statements). Except for the last statement, there is no control-transferring statement in it (in assembly programs: `jmp`, `call`, `ret`; in high-level languages: end of a branching, end of a loop, end of a procedure, `goto`). The statement sequence cannot be extended without violating the above two rules.

If the optimization happens within the bounds of basic blocks, then it is guaranteed that the transformation has no side effect. This is *local optimization*.

Peephole optimization

This is a method for certain kinds of local optimization. We examine only a part of a few statements from the code at a time. We compare the examined part with predefined patterns. If it matches, then we transform it according to the rule given for the pattern, and slide this “window” through the program. The transformations are given:

with a rule set $\{ \text{pattern} \rightarrow \text{replacement} \}$ (parameters can also be used in the pattern)

Examples:

- deletion of unnecessary operations: adding or subtracting zero
- simplifications: instead of multiplying by zero, clearing the register
- in the case of loading into a register and writing back to the same place, the write-back can be omitted
- deletion of statement repetitions: if possible, deletion of repetitions

7.2 Global optimization

The structure of the entire program has to be examined. Its method is data-flow analysis:

- The values of which variables does a given basic block compute?
- The values of which variables does which basic block use?

This makes possible the elimination of the multiple computation of identical expressions, even if they appear in different basic blocks; as well as the further propagation of constants and variables between basic blocks, and the optimization of branchings and loops.

8 Comparison of sequential and parallel/distributed execution

8.1 Sequential execution:

In this case the execution happens on one processor. Every operation is atomic. To one input belongs one output. Two sequential programs are equivalent if these pairs coincide. It does not use up all the available resources.

8.2 Parallel execution:

The program is executed on several processors. The parallel processes solve the given problem in cooperation, in synchrony, with each other. The concurrent program can be broken into elementary sequential programs; these are the processes. The processes can use common resources: e.g. variables, data type objects, communication channels.

The communication is usually realized in two ways.

With shared memory.

Then it has to be synchronized who accesses when, so that there is no collision.

With a communication channel.

It has to be guaranteed that if a process sends a message to another, then it also receives it, and also signals back. Care has to be taken so that no deadlock arises.

9 Representation of basic types

TODO

9.1 Array

An array is a data structure formed by a group of named elements, which can be referred to by their sequence number (index). It is also called a vector if one-dimensional, and sometimes a matrix if multidimensional. In most programming languages every single element has the same data type and the array is located continuously in the computer's memory. Based on the manner of creation it can be:

- static : the size is fixed, controlled in the declaration
- dynamic array: its size changes, it can be continuously extended

9.2 Record

A record is a construction usable for describing composite values. It has components provided with a name and a type; these are called fields. Its value set is the direct product of the base sets determined by the value types of the fields.



Figure 5: The general syntax of the record-type constructions.

Operations:

- Selection function (.fieldSelector syntax);
- construction function (RecordType(fieldValues) syntax);
- transformation functions are conceivable, which affect the whole record structure.

Example of a record and its use in C:

```
//definition of Point
struct Point
{
    int xCoord;
    int yCoord;
};

const struct Point ORIGIN = {0,0};
```

9.3 Class

A class is a user-defined type, on the basis of which instances (objects) can be created. A class essentially contains attribute and method (operation) definitions. The class describes the type of the object: it specifies its properties and their possible values (i.e. the type values), as well as the operations performable on the object (type operations).

Example in C++:

```
//definition of Point
class Point {
private:
    int xCoord;
    int yCoord;
public:
    //constructor
    Point(int xCoord, int yCoord) : xCoord(xCoord),yCoord(yCoord) {}

    void Translate(int dx, int dy) {
        xCoord+=dx;
        yCoord+=dy;
    }

    void Translate(Point delta) {
        xCoord+=delta.xCoord;
        yCoord+=delta.yCoord;
    }

    int getX() { return xCoord; }

    int getY() { return yCoord; }
};

int main(int argc, char* argv[])
```

```

{
    Point point(0,0);
    point.Translate(5,-2);
    cout<<point.getX()<<" "<<point.getY()<<endl; //5,-2
    Point delta(-2,1);
    point.Translate(delta);
    cout<<point.getX()<<" "<<point.getY()<<endl; //3,-1

    return 0;
}

```

Remark: Non-object-oriented languages do not have classes, e.g. older languages such as C, ADA, Pascal, or functional languages (Haskell, Clean, etc.).

9.4 Inheritance

A class can most simply be created by listing its data members and methods. However, the object-oriented paradigm also offers another, more efficient method: inheritance. With reusability in mind, inheritance makes it possible to create a new (child, descendant, sub-) class starting from an already existing (parent, ancestor, super-) class. Inheritance is a relationship between two classes in which the descendant class has almost all the properties of the parent class (it treats its non-private data members and methods as its own), and can supplement these with new ones. The class created this way can itself be the ancestor of other classes (except, for example, in Java classes marked with the **final** keyword, from which we can no longer derive), so these classes are organized into an inheritance hierarchy. Depending on whether a class has one or more ancestors, we speak of single or multiple inheritance. Java supports single inheritance, but for example C++ has multiple inheritance.

In Java the topmost element of the class hierarchy is the `Object` class, from which every other class (directly or indirectly) derives.

A subclass can reimplement the inherited methods. In this case the given method is realized with the same name but a different, modified (subclass-specialized) content. Such methods are called polymorphic. In Java every method that is not marked with the **final** keyword can be redefined. (In C++ everything that is not private.)

Examples:

- C++:

```

class RegularPolygon
{
protected:
    const double radius;
public:
    RegularPolygon(double radius) : radius(radius) {}
    virtual double area() = 0;
};

class EquilateralTriangle : public RegularPolygon
{
public:
    EquilateralTriangle(double radius) : RegularPolygon(radius) {}
    virtual double area()
    {

```

```
        return 0.75*sqrt(3.0)*radius*radius;
    }
};
```

- Java:

```
public abstract class RegularPolygon{
    protected final double radius;

    public RegularPolygon(double radius){
        this.radius = radius;
    }
    public abstract double area();
}

public class EquilateralTriangle extends RegularPolygon{
    public EquilateralTriangle(double radius){
        super(radius);
    }
    public double area(){
        return 0.75*Math.sqrt(3.0)*radius*radius;
    }
}
```

10 Scope/visibility

1. Scope: At declaration the programmer connects an entity (for example a variable or function) with a name. By scope we mean that part of the source text while this connection is in effect. This generally lasts to the end of the block that contains the given declaration.
2. Visibility is a subset of the scope, that part of the program text where the given entity belongs to the declared name. Since in nested blocks we can connect an already previously introduced name to another entity, in such a case the entity declared in the outer block is no longer reachable by its name. We call this the shadowing of visibility.

In some languages (for example C++) in certain cases (for example class-level data members) the entity declared in the outer block can be accessed by qualified name even then.

11 Automatic, static and dynamic lifetime, garbage collection

Lifetime: by the lifetime of variables we mean the section of the program's execution time during which the storage allocated for the variable belongs to the variable.

11.1 Automatic lifetime

Local variables declared inside blocks have automatic lifetime, which means it lasts from the declaration to the end of the containing block, i.e. it coincides with the scope. Storage for them is allocated in the current activation record of the execution stack.

11.2 Static lifetime

Global variables, and in some languages variables declared as static (for example in C/C++ with the `static` keyword), have static lifetime. The lifetime of such variables extends over the entire execution time of the program; storage for them can already be allocated at compile time.

11.3 Dynamic lifetime

For variables with dynamic lifetime the programmer allocates storage for them on the dynamic storage area (heap), and it is also the programmer's task to ensure that this storage area is later released. If they forget about the latter, this is called a memory leak. As we can see, in the case of dynamic lifetime the scope is not connected to the lifetime in any way; the lifetime can be narrower or wider than the scope.

11.4 Garbage collector

The other name of the garbage collector is the waste collector; after the English name Garbage Collector it is often abbreviated to just GC. Its task is to automate the storage release associated with dynamic memory management, and to take the responsibility off the programmer's shoulders, thereby reducing the chance of errors.

The garbage collector observes which variables have gone out of their scope, and declares them releasable. The disadvantages of the method are its computational cost and its nondeterminism. The garbage collector does not release variables that have gone out of scope immediately, and the order of release is not the same as the order in which the variables became releasable.

When and which variable the garbage collector releases is determined by a complex algorithm that is specific to each programming language, in which the saturation of the available memory and the estimated time needed for release usually play a role. (For example, if an object has a destructor, then the GC usually releases it only later.)

Overall, the garbage collector only guarantees that sooner or later (at the latest by the end of the program's run) it releases every dynamically allocated variable.

Languages that use garbage collection are e.g. Java, C#, Ada. In C/C++ there is no garbage collection; the programmer has to take care of releasing the dynamically allocated memory areas.

12 Representation of variables in memory

TODO

13 Subprograms, parameter passing, overloading

13.1 Subprograms

We call functions, procedures, and operations subprograms. With their help the program can be divided by tasks; the independent tasks can be outsourced from the main program.

13.2 Parameter passing

Subprograms may need input data and can also return values. We write subprograms generally, with their own variable names; these are the formal parameters. When the subprogram is called, the formal parameters receive values based on the passed actual parameters. What the value of the formal parameters will be depends on the manner of parameter passing.

13.2.1 Textual parameter passing

Used in macros to this day. In the body of the macro, the text of the actual parameter is written in place of the formal parameter.

13.2.2 Call by name

The actual-parameter expression is re-evaluated each time a reference is made to the formal one. We evaluate the parameter in the context of the body, so different occurrences of the formal parameter can mean different things within the subprogram. Its use is archaic (for example: Algol 60, Simula 67).

13.2.3 Call by value

One of the most widespread parameter-passing modes (for example: C, C++, Pascal, Ada, Java), with input semantics. The formal parameter is a local variable of the subprogram; at the call a copy of the actual parameter is made on the stack, this will be the formal one. At the end of the subprogram the formal parameter ceases to exist.

13.2.4 Call by reference (by address)

The other most widespread parameter-passing mode (for example: Pascal, C++, C#), with input and output semantics. At the call the address of the actual parameter is passed, that is, the formal and the actual parameter mean the same object, an alias is created.

Remark: In Java there is no call by reference, only by value. Java, namely, stores the value of the primitive types, and for objects a reference to the given object (as the reference type in C++). When passing an object, this reference is copied, that is, the reference is passed by value.

For example:

```
public static void BadSwap(Object x, Object y)
{
    Object tmp = x;
    x = y;
    y = tmp;
}
```

The above function does not swap x and y, only locally (within the function body), but at the call site x and y both stay in place.

13.2.5 Call by result

A parameter-passing mode with output semantics. The formal parameter is a local variable of the subprogram; at the end of the subprogram the value of the formal parameter is copied into the actual one. However, when the subprogram is called, the value of the actual one is not copied into the formal one. (Used, for example, by Ada.)

13.2.6 Call by value/result

The combination of call by value and call by result, so we obtain a parameter-passing mode with input and output semantics. (Used, for example, by Algol-W or Ada.)

13.2.7 Call by sharing

The parameter-passing mode of object-oriented programming languages (for example: CLU, Eiffel). Its essence is that if the formal parameter is mutable and the actual parameter is a mutable variable, then call by reference

happens, otherwise call by value. The manner of parameter passing cannot be separately specified. By a mutable object we mean that its properties, and thereby its state, can be changed.

13.2.8 Call by need

This parameter-passing mode is applied by lazy-evaluation functional languages (for example: Clean, Haskell, Miranda). It evaluates the actual parameter not at the call, but when it first needs it for the computations.

13.3 Overloading

With the help of overloading we can create subprograms with the same name but different signatures. In most programming languages the signature means the name of the subprogram together with the number and types of the formal parameters, but in some languages (for example Ada) the type of the return value also belongs to it. The primary use of overloading is to be able to perform the same activity with different parameterizations.

From the subprogram call, the compiler can decide which of the overloaded variants should be invoked. If none matches or several match, a compile error occurs.

14 Exception handling

Exception handling is a programming mechanism whose goal is to handle an event (error) or statement that interrupts the run of the program intentionally or unintentionally. The event itself is called an exception. It can be used for error handling reaching beyond the traditional, sequential and structured programming frame, as well as for higher-level error detection, possibly correction.

So that errors do not remain unhandled, it is worth requiring error handling at the language level. The essence is that when an error appears in the program, that is, an exceptional event happens, the normal execution of the program stops, and control is handed over to the exception handling mechanism. Since the basic philosophy is anyway that the erroneous code cannot work well until the error is handled, it is not worth continuing the execution of the program. By the fact that the code handling the exception is separated from the normally running code, we are also given the advantage that the code itself becomes cleaner and more transparent; we do not have to interrupt the run of the code with all sorts of checks just to check whether there is an error in it.

14.1 Exceptions

An exceptional condition is a problem that prevents the running of the code of a current method or scope. It is important to distinguish the exceptional condition from a normal problem, in which, in the given situation, there is enough information to overcome the problem. When an exceptional condition arises, we cannot normally continue the execution of the program, because there is not enough information regarding how we could remedy the problem. What we can do is jump out of the given context and entrust the solution of the problem to a broader environment. Exactly this happens when we throw an exception.

For example, such a case can be division. What if we just tried to divide by 0? It is conceivable that for a given problem we know how to handle division by 0, but if this is an unexpected value in the given situation, then it may be more advisable to throw an exception instead of continuing the execution of the program on the given execution path.

```
if (p < 0)
    throw new IllegalArgumentException();
```

When we throw an exception, several things happen. First, we create an exception object just like any other object, with `new`, which after this, like any object, is created on the heap. The execution of the program (which, of course, cannot now continue because of the arising of the exception) stops, and the reference pointing to the exception object exits the current execution context through the `throw` statement. At this point the exception handling mechanism takes over the execution, and tries to find the place where the run of the program can be

continued. This appropriate place is the exception handler, which handles the problem causing the given exception, and then the program can continue there, or precisely generates another exception.

14.2 Arguments of exceptions

Every built-in exception has two constructors: a default one and one that expects a String in its parameter (that String will initialize the message data member of the exception, which we can later even use).

```
if (p < 0)
    throw new IllegalArgumentException("p cannot be negative!");
```

The throw keyword throws the created exception onward. If the exception is not handled in the given method, then it is as if throw were an alternative return statement of the method, whose return value, of course, is not of the type that the method's declaration originally promises. It may also be that even the calling method does not handle the given exception; in such a case the thrown exception is passed along through the methods of the call stack until it reaches the appropriate exception handler, or reaches the bottom of the call stack. Generally, though, we can expect that if an exception is thrown somewhere, then there will be a place where someone handles it (if we do not write our programs this way, then their usability will quite quickly be called into question).

14.3 Catching an exception

Those parts of the program where exceptions can arise, and which are then followed by exception handler parts that will handle the arising exceptions, are called the protected regions of the program.

```
try {
    // normal code in which an exception can arise
    // (protected region)
} catch (ExceptionType1 e1) {
    // error handling code for exception e1
} catch (ExceptionType2 e2) {
    // error handling code for exception e2
    throw e2; // can be rethrown
} catch (Exception e) {
    // error handling code for all remaining exceptions
} finally {
    // finally ( always runs )
}
```

When throwing an exception, we can throw any exception that derives from Throwable (the ancestor exception class). The error handler blocks do not necessarily handle all exception types. The information about the error is generally contained in the exception object, implicitly already in the name of the exception class itself, so in a broader context of the error it can be decided which exceptions can be handled at a given point.

14.4 Java standard exceptions

As was already mentioned, the ancestor exception class is the Throwable class. It has two direct descendants. One is the Error class, with which one generally does not have to deal, because it represents errors appearing at compile time, or system errors. The Exception class is the one that is practically the ancestor class of the exceptions used in programs. What exceptions are used within the Java libraries we can read about in the specification of the appropriate Java version.

14.5 Runtime exception

Runtime exceptions are special Java standard exceptions that do not have to be separately specified in the exception specification. It will be the ancestor of those exceptions that the virtual machine can throw during normal operation. For example, these are `NullPointerException`, `ClassCastException`, `IndexOutOfBoundsException`. The reason these exceptions do not have to be shown in the specification of the methods is that almost every method can be assumed to be able to throw such exceptions.